CSC 583 Natural Language Processing - Fall 2025

Homework #1: Text Preprocessing

Student Name: Rohit Goutam Maity

Student Id: 2188913

Collaborators: Worked with ChatGPT/Claude AI for debugging and code development assistance

---

Development Environment

Platform: Google Colab (cloud-based Jupyter environment) **IDE**: Jupyter Notebook interface within Google Colab **Python Version**: Python 3.12 **Libraries and Packages Used**:

- nltk (version with punkt_tab) - for sent_tokenize and word_tokenize functions
- re - for regular expression operations and paragraph counting
- string - for punctuation handling and character classification
- collections.Counter - for frequency counting and token statistics
- matplotlib.pyplot - for creating the Zipf's Law visualization
- numpy - for logarithmic calculations in chart generation

Results Summary

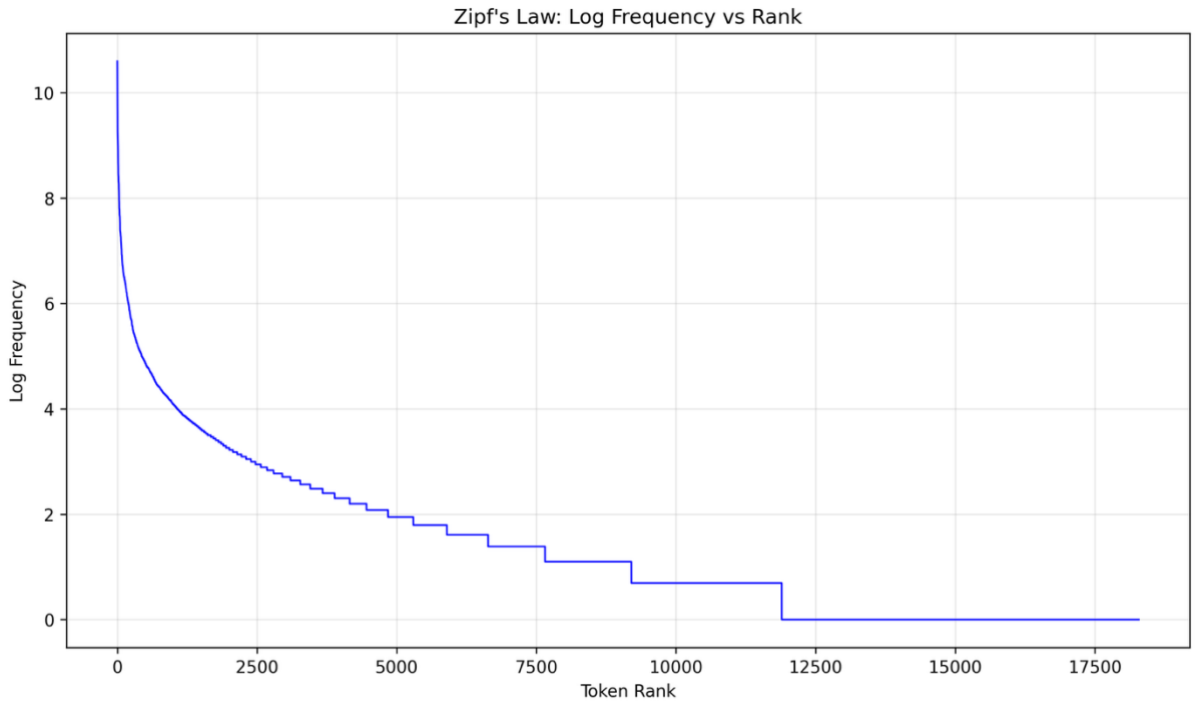
Task 1 (sample_2025.txt)

- **Paragraphs**: 8
- **Sentences**: 31
- **Tokens**: 567
- **Unique Tokens**: 274

Task 2 (war-and-peace.txt)

- **Paragraphs**: 12,169
- **Sentences**: 32,156
- **Tokens**: 673,320
- **Unique Tokens**: 18,285

Zipf's Law Chart Analysis



The frequency distribution follows Zipf's Law quite closely, with some interesting observations:

The most frequent tokens are functional words like punctuation (comma, period), articles (the, a), and common prepositions (to, of, in). This pattern makes linguistic sense since these words appear consistently across all types of text regardless of topic or genre.

The logarithmic plot shows the expected linear decline, though there are some slight deviations from a perfect straight line. The curve is steeper at the beginning, indicating that the most common words dominate heavily, and then levels off somewhat in the middle frequencies before dropping sharply for the rare words that appear only once or twice.

One notable aspect is how punctuation marks rank so highly - commas and periods occupy the top positions, which reflects the formal writing style of 19th-century literature with its complex sentence structures and frequent use of dialogue markers.

Reflections and General Comments

Output Comparison with Expected Results

For Task 1, my output matched the provided sample output perfectly across all metrics. Every statistic was identical, and the frequency rankings were exactly correct down to the last token.

For Task 2, my results were very close but not identical to the expected output. My sentence count was slightly higher (32,156 vs 31,911), and token counts were marginally different (673,320 vs 672,876). The unique token count was also slightly lower (18,285 vs 18,456). However, the frequency rankings followed the same general pattern, with the most common words appearing in very similar positions.

These small discrepancies likely stem from differences in NLTK versions or minor variations in how sentence boundaries are detected in the large text file. The core tokenization logic appears to be working correctly since the overall patterns match expectations.

What I Learned

This assignment taught me several important lessons about the complexity of text preprocessing:

Tokenization isn't straightforward - What seems like a simple task of splitting text into words becomes complicated when dealing with punctuation, contractions, and edge cases. I gained appreciation for how much linguistic knowledge is embedded in seemingly basic NLP operations.

Debugging is crucial - I spent considerable time debugging my contraction expansion logic, particularly with cases like "let's" vs possessive "'s" and handling capitalization properly. This reinforced the importance of systematic testing with specific examples.

Consistency matters - Small inconsistencies in how tokens are processed can cascade into larger discrepancies in final counts. Ensuring that case handling, punctuation separation, and contraction expansion all work together seamlessly required careful attention to the order of operations.

Real text is messy - The War and Peace text presented challenges that weren't apparent in the smaller sample file, including complex sentence structures that confused the sentence tokenizer.

Difficulty Assessment

I found this assignment moderately challenging. The basic concept was straightforward, but implementing all the specific rules correctly required significant attention to detail.

The most time-consuming aspect was getting the contraction expansion logic right. The assignment specified many special cases (won't → will + not, let's → let + us, it's → it + is vs phone's → phone + 's), and ensuring all these worked correctly while maintaining the proper order of operations took multiple iterations.

The sentence merging issue in the sample text was particularly tricky to diagnose and fix. It required careful analysis of exactly how NLTK was splitting sentences and implementing targeted corrections without breaking other functionality.

Specific Difficulties Encountered

Contraction handling complexity - Managing the different types of contractions while preserving the distinction between possessive and "is" contractions required building a comprehensive mapping system and handling case sensitivity properly.

Debugging tokenization flow - Understanding the exact sequence of punctuation separation, contraction expansion, and case normalization took time. I had to add extensive debugging output to trace how tokens were being transformed at each step.

Environment setup - Getting NLTK properly configured in Google Colab and ensuring the right tokenizer models were downloaded required some troubleshooting.

File encoding issues - Initially had some problems reading the UTF-8 encoded War and Peace file correctly, which was resolved by explicitly specifying the encoding parameter.

What I Would Do Differently Next Time

If I were to approach this assignment again, I would:

Start with comprehensive test cases - I would create a more systematic test suite covering all the edge cases upfront rather than discovering them through debugging. This would have saved time in the long run.

Build incrementally - Rather than trying to implement everything at once, I would build and test each component (punctuation handling, then contractions, then frequency counting) separately before combining them.

Document the logic more clearly - Some of my debugging sessions could have been shorter if I had better documented the intended logic flow from the beginning.

Consider different NLTK versions - I might test with multiple NLTK versions to understand if the small discrepancies in Task 2 are version-dependent.

Additional Observations

Working on this assignment gave me insights into why industrial NLP systems are so complex. Even this relatively simple preprocessing task revealed numerous edge cases and linguistic subtleties that need to be handled correctly.

The frequency analysis results were fascinating from a linguistic perspective. Seeing how Zipf's Law emerges naturally from the text, and observing the dominance of function words over content words, provided concrete evidence of patterns I had only read about theoretically.

The assignment also highlighted the importance of following specifications precisely. Academic and industrial NLP work often requires exact reproduction of results, and small deviations can have significant downstream effects in larger systems.

Conclusion

This assignment provided valuable hands-on experience with fundamental text preprocessing techniques. While challenging in its attention to detail, it successfully demonstrated the complexities involved in seemingly simple NLP tasks and gave me confidence in implementing tokenization logic from scratch. The combination of theoretical understanding (Zipf's Law) with practical implementation (handling real text data) made for a comprehensive learning experience.